

Savol-javob

1. Model Architecture nima?

Model Architecture — modelning ichki tuzilishi, qismlari va ma'lumot model ichida qanday harakatlanishini ko'rsatadigan umumiy qurilish hisoblanadi. Architecture so'zining o'zbekcha ma'nosi "arxitektura", "tuzilish", "qurilish" degani. Deep Learning va Computer Visionda architecture model qanday qatlamlardan iborat bo'lishini, input qayerdan kirishini, qaysi qatlamlarda qayta ishlanishini va output qayerdan chiqishini bildiradi.

Model Architecture modelning tanasi yoki skeleti kabi tushuniladi. Inson tanasida ko'z, miya va qaror chiqarish jarayoni bo'lgani kabi, Computer Vision modelida ham input, processing va output qismlari bo'ladi.

Oddiy ko'rinishda architecture quyidagicha tushuniladi:

Input → Model → Output

Computer Vision uchun esa bu yanada aniqroq ko'rinishda bo'ladi:

Input Image → Backbone → Neck → Head → Output

Input Image — modelga kiradigan rasm. Image — rasm degani. Kompyuter rasmni inson kabi "mushuk", "it" yoki "mashina" deb ko'rmaydi. U rasmni pixel qiymatlari orqali ko'radi. Pixel — rasmning eng kichik nuqtasi. Har bir pixel rang yoki yorug'lik qiymatiga ega bo'ladi.

Backbone — modelning asosiy feature ajratadigan qismi. Backbone soʻzining oddiy maʼnosi “umurtqa pogʻonasi” yoki “asosiy tayanch” degani. Modelda backbone rasm ichidagi muhim belgilarni ajratib oladi. Feature — belgi yoki xususiyat. Masalan, mushukni tanishda quloq, koʻz, moʻylov, yuz shakli feature boʻlishi mumkin.

Neck — backbone va head orasidagi bogʻlovchi qism. Neck soʻzining oddiy maʼnosi “boʻyin”. Modelda neck backbone ajratgan featurelarni tartiblaydi, ixchamlaydi yoki keyingi qismga mos formatga keltiradi.

Head — modelning yakuniy qaror chiqaradigan qismi. Head soʻzining oddiy maʼnosi “bosh”. Image Classificationda head class prediction chiqaradi. Class — kategoriya yoki tur. Masalan, cat, dog, horse, deer.

Output — modeldan chiqadigan yakuniy natija. Masalan:

Cat: 85%

Dog: 15%

Bu natijada model rasmni 85 foiz ehtimol bilan mushuk deb hisoblaydi. Probability — ehtimollik degani.

Model Architecture quyidagi savollarga javob beradi:

Model qanday input qabul qiladi?

Input qaysi qatlamlardan oʻtadi?

Featurelar qayerda ajratiladi?

Maʼlumot qayerda tartiblanadi?

Yakuniy qaror qayerda chiqadi?

Output qanday koʻrinishda boʻladi?

Masalan, Image Classification modelida rasm avval input sifatida olinadi. Keyin convolutional layerlar orqali rasm ichidan qirra, chiziq,

rang o'zgarishi, shakl kabi belgilar ajratiladi. Keyin pooling layerlar orqali ma'lumot ixchamlanadi. So'ng fully connected layer classlar bo'yicha score chiqaradi. Softmax esa shu scorelarni ehtimollikka aylantiradi.

Convolutional Layer — rasm ichidagi featurelarni filterlar yordamida ajratadigan qatlam. Filter — rasm ustida yurib, kerakli belgilarni topadigan kichik matematik oynacha.

Pooling Layer — muhim ma'lumotlarni saqlagan holda feature map hajmini kamaytiradigan qatlam. Feature map — model ajratgan belgilar xaritasi.

Fully Connected Layer — yakuniy qaror chiqarishda ishlatiladigan zich bog'langan qatlam. Dense layer ham shu ma'noda ishlatiladi. Dense — zich bog'langan degani.

Softmax — model chiqargan class scorelarni ehtimollikka aylantiradigan activation function. Activation function — modelga murakkab patternlarni o'rganishga yordam beradigan funksiya.

Pattern — takrorlanadigan belgi yoki qonuniyat.

Architecture yaxshi qurilsa, model rasm ichidan kerakli featurelarni yaxshi ajratadi. Architecture noto'g'ri yoki juda sodda bo'lsa, model murakkab belgilarni o'rgana olmaydi. Masalan, mushuk va itni ajratish uchun model quloq, ko'z, yuz, tana, rang va boshqa belgilarni tushunishi kerak. Juda sodda model bu belgilarni yetarli o'rgana olmasligi mumkin.

Lekin architecture borligi model o'rgandi degani emas. Architecture faqat modelning tuzilishini bildiradi. Model o'rganishi uchun training kerak bo'ladi. Training — modelni data orqali o'qitish jarayoni.

Optimization — model xatosini kamaytirib, weightlarni yangilash jarayoni.

Weight — model ichidagi o‘rganiladigan vazn qiymati. Bias — model hisoblashiga qo‘shimcha moslashuv beradigan parametr. Weight va bias training davomida o‘zgaradi.

Model Architecture qisqa qilib shunday tushuniladi:

Architecture = modelning ichki tuzilishi

Training = modelni o‘qitish jarayoni

Optimization = model xatosini kamaytirish jarayoni

Model architecture model “qanday qurilgan?” degan savolga javob beradi. Training esa model “qanday o‘rganadi?” degan savolga javob beradi.

2. Image Classification model yasash uchun asosiy 4 bosqich

Image Classification model yasash uchun asosiy 4 bosqich quyidagicha bo‘ladi:

1. Data Preparation
2. Model Architecture
3. Training & Optimization
4. Deployment & Inference

Bu to‘rt bosqich modelni 0 dan boshlab real ishlaydigan holatgacha olib boradigan asosiy yo‘l hisoblanadi.

1-bosqich: Data Preparation

Data Preparation — ma'lumotni tayyorlash. Bu bosqich Image Classification projectining eng muhim qismlaridan biri hisoblanadi. Model qanday data ko'rsa, shundan o'rganadi. Agar data noto'g'ri bo'lsa, model ham noto'g'ri o'rganadi.

Data — ma'lumot. Image Classificationda data rasmlar va ularning labellaridan iborat bo'ladi. Label — rasmning to'g'ri javobi. Masalan, mushuk rasmi uchun label `cat`, it rasmi uchun label `dog`.

Data Preparation ichida quyidagi ishlar bajariladi:

Data Acquisition

Preprocessing

Augmentation

Train / Validation / Test split

Data Acquisition — rasmlarni yig'ish va label qilish. Acquisition — yig'ish yoki olish degani. Masalan, `cat` va `dog` classification qilish uchun mushuk va it rasmlari yig'iladi.

Preprocessing — rasmni modelga mos holatga keltirish. Bunda `resize`, `normalize`, `crop` kabi ishlar bajariladi.

Resize — rasm o'lchamini o'zgartirish. Masalan, barcha rasmlar `224 x 224` o'lchamga keltiriladi.

Normalize — pixel qiymatlarini modelga qulay oraliqqa keltirish. Oddiy rasmda pixel qiymatlari 0 dan 255 gacha bo'ladi.

Normalizationdan keyin ular 0 dan 1 gacha yoki `mean/std` asosida boshqa scale'ga o'tishi mumkin. Mean — o'rtacha qiymat. Std — standard deviation, ya'ni standart og'ish.

Crop — rasmning kerakli qismini kesib olish.

Augmentation — mavjud rasmlardan turli ko‘rinishlar yaratish. Masalan, rasmni aylantirish, chap-o‘ngga ag‘darish, yorug‘likni o‘zgartirish, contrastni o‘zgartirish. Brightness — yorug‘lik. Contrast — ranglar orasidagi farq.

Augmentation modelni kuchliroq qiladi, chunki model obyektni har xil holatda ko‘rishga o‘rganadi. Bu generalizationni yaxshilaydi. Generalization — modelning oldin ko‘rmagan yangi data’da ham yaxshi ishlashi.

Train / Validation / Test split — datasetni uch qismga ajratish.

Train — model o‘rganadigan qism.

Validation — training davomida modelni tekshirish uchun ishlatiladigan qism.

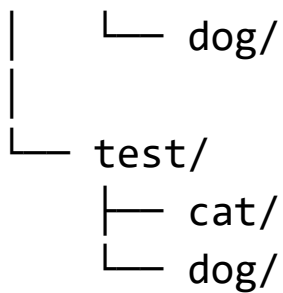
Test — training tugagandan keyin yakuniy baholash uchun ishlatiladigan qism.

Masalan:

70% train
15% validation
15% test

Image Classification dataset structure odatda quyidagicha bo‘ladi:

```
data/  
├─ train/  
│   ├── cat/  
│   └── dog/  
└─ val/  
    └─ cat/
```



Bu structure folder-based labeling deyiladi. Folder-based labeling — papka nomiga qarab label berish usuli. Masalan, `cat` papkasidagi barcha rasmlar `cat` labelini oladi.

2-bosqich: Model Architecture

Model Architecture — modelning tuzilishini yaratish. Bu bosqichda model qanday qatlamlardan iborat bo‘lishi tanlanadi.

Image Classification uchun model structure odatda quyidagicha bo‘ladi:

Input Image



Backbone



Neck



Head



Output

Input Image — modelga kiradigan rasm.

Backbone — feature extraction qiladigan asosiy qism. Feature extraction — rasm ichidan muhim belgilarni ajratish. Backbone

sifatida CNN, ResNet, EfficientNet, MobileNet, Vision Transformer kabi modellar ishlatilishi mumkin.

CNN — Convolutional Neural Network. O‘zbekcha ma’nosi “konvolyutsion neyron tarmoq”. CNN rasm ichidagi qirra, chiziq, shakl va obyekt qismlarini ajratishda ishlatiladi.

ResNet — Residual Network. Chuqur CNN modeli. Residual connection orqali chuqur qatlamlarda o‘rganish osonlashadi.

EfficientNet — anqlik va tezlikni muvozanat qiladigan model oilasi.

MobileNet — yengil model. Telefon va edge device uchun qulay. Edge device — kichik qurilma, masalan telefon, kamera yoki Raspberry Pi.

Vision Transformer yoki ViT — rasmni patchlarga bo‘lib tahlil qiladigan transformer model. Patch — rasmning kichik bo‘lagi. Transformer — attention mexanizmiga asoslangan model. Attention — modelning muhim joylarga ko‘proq e’tibor berishi.

Neck — backbone ajratgan featurelarni tartiblaydi. Image Classificationda neck sifatida Global Average Pooling ishlatilishi mumkin. Global Average Pooling — feature mapni vector ko‘rinishiga keltiradigan amal.

Head — yakuniy class prediction chiqaradi. Fully Connected Layer va Softmax ko‘pincha head qismida ishlatiladi.

Softmax class scorelarni probabilityga aylantiradi. Masalan:

Cat: 0.85

Dog: 0.15

Bu model rasmni 85% ehtimol bilan cat deb topganini bildiradi.

3-bosqich: Training & Optimization

Training & Optimization — modelni o‘qitish va xatosini kamaytirish jarayoni. Bu bosqichda model data orqali o‘rganadi.

Training jarayoni quyidagicha ishlaydi:

Input → Model → Prediction → Loss → Backpropagation
→ Optimizer Step → Better Model

Input — modelga kiradigan rasm yoki batch.

Batch — datasetning kichik qismi. Model har safar butun datasetni emas, kichik batchni ko‘radi.

Prediction — model taxmini. Masalan, model rasmni **cat** yoki **dog** deb topadi.

Loss — model qanchalik xato qilganini ko‘rsatadigan qiymat. Loss katta bo‘lsa, model noto‘g‘ri prediction qilgan bo‘ladi. Loss kichik bo‘lsa, model to‘g‘ri javobga yaqin bo‘ladi.

Loss Function — lossni hisoblaydigan funksiya. Image Classificationda Cross-Entropy Loss ko‘p ishlatiladi. Cross-Entropy Loss model haqiqiy classga qanchalik probability berganiga qarab xatoni hisoblaydi.

Backpropagation — xatoni orqaga tarqatish. Bu jarayon model ichidagi har bir weight qanchalik o‘zgarishi kerakligini aniqlaydi.

Gradient — lossni kamaytirish uchun weight qaysi yo‘nalishda o‘zgarishi kerakligini ko‘rsatadigan qiymat.

Optimizer — gradientlardan foydalanib weightlarni yangilaydigan algoritm. Masalan, SGD, Adam, AdamW.

Learning Rate — weight update qadamining kattaligi. Learning rate juda katta bo'lsa, model beqaror bo'ladi. Juda kichik bo'lsa, model juda sekin o'rganadi.

Epoch — model butun training datasetni bir marta to'liq ko'rib chiqishi.

Training loop odatda quyidagicha bo'ladi:

Forward Pass

↓

Loss hisoblash

↓

Backward Pass

↓

Optimizer Step

↓

Takrorlash

Forward Pass — input modeldan oldinga qarab o'tib prediction chiqaradi.

Backward Pass — lossdan boshlab gradientlarni orqaga hisoblaydi.

Optimizer Step — weightlarni yangilaydi.

4-bosqich: Deployment & Inference

Deployment — modelni real ishlaydigan holatga chiqarish. Inference — train qilingan model yordamida yangi rasmga prediction qilish.

Trainingda model o'rganadi. Inference bosqichida model tayyor bilimidan foydalanadi.

Inference jarayoni:

New Image

↓

Preprocessing

↓

Trained Model

↓

Prediction

↓

Result

New Image — yangi rasm.

Preprocessing — rasmni trainingdagi formatga moslash. Masalan, resize va normalization.

Trained Model — oldin train qilingan model.

Prediction — model taxmini.

Result — foydalanuvchiga ko‘rsatiladigan natija.

Deployment turlari:

API

Web app

Mobile app

Edge device

Cloud server

API — Application Programming Interface. Boshqa dasturlar modelga so‘rov yuborishi va natija olishi uchun interfeys.

Web app — brauzer orqali ishlaydigan ilova.

Mobile app — telefon ilovasi.

Cloud server — internetdagi server.

Edge device — kichik qurilma ichida ishlaydigan model.

Deploymentda modelni tezroq va yengilroq qilish ham muhim.
Buning uchun Quantization, ONNX, TensorRT ishlatilishi mumkin.

Quantization — modelni yengillashtirish uchun float32 kabi katta son formatlarini int8 kabi kichik formatga o'tkazish.

ONNX — modelni turli frameworklarda ishlatishga yordam beradigan format.

TensorRT — NVIDIA GPUda inference tezligini oshirish vositasi.

Monitoring ham deploymentning muhim qismi. Monitoring — model productda ishlayotganda uning natijasi, tezligi, xatolari va data driftni kuzatish. Data drift — real hayotdagi data training data'dan farq qilib ketishi.

Image Classification model yasashning 4 bosqichi qisqa ko'rinishda:

1. Data Preparation → rasm va label tayyorlash
2. Model Architecture → model tuzilishini yaratish
3. Training & Optimization → modelni o'qitish va xatoni kamaytirish
4. Deployment & Inference → modelni real ishlatish

3. Backbone, Neck, Head nima vazifa bajaradi?

Backbone, Neck va Head Computer Vision modelining asosiy qismlari hisoblanadi. Bu uch qism model ichida turli vazifa bajaradi. Ularni inson tanasi nomlari orqali eslab qolish oson: backbone — umurtqa pogʻonasi, neck — boʻyin, head — bosh.

Modelda bu uch qism quyidagicha ishlaydi:

Input Image → Backbone → Neck → Head → Output

Backbone vazifasi

Backbone — modelning feature extraction qiladigan asosiy qismi. Backbone soʻzining oddiy maʼnosi “umurtqa pogʻonasi” yoki “asosiy tayanch”. Modelda backbone rasm ichidan kerakli featurelarni ajratib oladi.

Feature — rasm ichidagi foydali belgi yoki xususiyat. Masalan:

qirra
chiziq
burchak
rang oʻzgarishi
koʻz
quloq
gʻildirak
barg dogʻi
hayvon tanasi

Feature extraction — shu belgilarni rasm ichidan ajratib olish jarayoni.

Backbone rasmni oddiy pixel qiymatlaridan murakkabroq featurelar ko'rinishiga o'tkazadi. Dastlabki qatlamlar oddiy belgilarni topadi. Keyingi qatlamlar murakkabroq qismlarni o'rganadi. Oxirgi qatlamlarda obyektga yaqin ma'noli featurelar hosil bo'ladi.

Masalan, mushuk rasmini tanishda:

Birinchi qatlamlar → qirra, chiziq, rang o'zgarishi
Keyingi qatlamlar → ko'z, quloq, mo'ylov
Oxirgi qatlamlar → mushuk yuziga o'xshash umumiy belgi

Backbone sifatida quyidagi modellar ishlatilishi mumkin:

CNN

ResNet

EfficientNet

MobileNet

Vision Transformer

CNN — rasm ichidagi lokal featurelarni filterlar orqali topadigan model.

ResNet — chuqur CNN, residual connection bilan ishlaydi.

EfficientNet — tezlik va accuracy muvozanatiga qaratilgan model.

MobileNet — yengil va tez model.

Vision Transformer — rasmni patchlarga bo'lib attention orqali tahlil qiladi.

Backbone yaxshi feature ajrata olsa, modelning yakuniy natijasi ham yaxshi bo'lishi mumkin. Agar backbone featurelarni yomon ajratsa, head yaxshi bo'lsa ham natija past bo'ladi.

Backbone qisqa qilib:

Backbone = rasm ichidan kerakli featurelarni
ajratadigan asosiy qism

Neck vazifasi

Neck — backbone va head orasidagi bog'lovchi qism. Neck so'zining oddiy ma'nosi "bo'yin". Modelda neck backbone ajratgan featurelarni head uchun tayyorlaydi.

Neckning asosiy vazifalari:

featurelarni tartiblash

featurelarni ixchamlashtirish

featurelarni birlashtirish

featurelarni head qismiga mos formatga keltirish

Image Classificationda neck odatda Global Average Pooling bilan ifodalanishi mumkin.

Global Average Pooling — feature mapni bitta vector ko'rinishiga keltiradigan operatsiya. Feature map — backbone ajratgan belgilar xaritasi. Vector — raqamlar ro'yxati.

Masalan, backbone chiqishi quyidagicha bo'lishi mumkin:

Feature map = $7 \times 7 \times 512$

Bu yerda:

7 → balandlik

7 → kenglik

512 → channel soni

Global Average Pooling har bir channel bo'yicha o'rtacha qiymat oladi va natijani quyidagicha ixchamlaydi:

$7 \times 7 \times 512 \rightarrow 1 \times 1 \times 512$

Bu head qismiga berish uchun qulayroq bo'ladi.

Object Detection va Segmentationda neck yanada muhimroq bo'ladi. U yerda FPN, upsampling, concatenation kabi qismlar ishlatiladi.

FPN — Feature Pyramid Network. Turli o'lchamdagi featurelarni birlashtirish uchun ishlatiladi.

Upsampling — feature map resolutionini kattalashtirish.

Concatenation — featurelarni birlashtirish.

Image Classificationda neck oddiyroq bo'lishi mumkin, lekin baribir backbone va head orasidagi ko'prik vazifasini bajaradi.

Neck qisqa qilib:

Neck = backbone ajratgan featurelarni tartiblab, headga tayyorlab beradigan qism

Head vazifasi

Head — yakuniy qaror chiqaradigan qism. Head soʻzining oddiy maʼnosi “bosh”. Modelda head class prediction chiqaradi.

Head backbone va neckdan kelgan featurelarga qarab classlar boʻyicha score hosil qiladi.

Masalan, model cat vs dog classification qilsa, head quyidagicha score chiqarishi mumkin:

Cat score = 3.2

Dog score = 1.1

Bu scorelar hali probability emas. Softmax orqali ular ehtimollikka aylantiriladi:

Cat = 85%

Dog = 15%

Head ichida odatda Fully Connected Layer va Softmax ishlatiladi.

Fully Connected Layer — featurelar orasidagi munosabatlarni oʻrganadigan qatlam. Dense Layer ham shu maʼnoda ishlatiladi.

Softmax — class scorelarni probabilityga aylantiradi.

Head yakuniy qaror chiqaradi:

Prediction: Cat

Confidence: 85%

Confidence — modelning ishonch darajasi.

Head qisqa qilib:

Head = featurelar asosida yakuniy class prediction chiqaradigan qism

Backbone, Neck, Head umumiy farqi

Backbone, Neck va Head bir-biri bilan ketma-ket ishlaydi:

Backbone → featurelarni ajratadi

Neck → featurelarni tartiblaydi

Head → yakuniy qaror chiqaradi

Oddiy hayotiy misol:

Backbone = ma'lumot yig'adigan ishchi

Neck = ma'lumotni tartiblaydigan bo'lim

Head = yakuniy qaror chiqaradigan rahbar

Image Classificationda:

Input rasm

↓

Backbone rasm ichidan belgilarni topadi

↓

Neck belgilarni ixchamlaydi

↓

Head class ehtimolliklarini chiqaradi

↓

Output class tanlanadi

Cat vs Dog misolida:

Backbone → quloq, koʻz, yuz shakli, moʻylov
featurelarini ajratadi

Neck → shu featurelarni tartiblab vector qiladi

Head → cat yoki dog classini tanlaydi

4. Model faqat architecture bilan oʻrganib qoladimi? Nima yetishmayapti?

Model faqat architecture bilan oʻrganib qolmaydi. Architecture modelning tuzilishini bildiradi, lekin oʻrganish jarayoni training va optimization orqali sodir boʻladi.

Architecture modelning “qanday qurilganini” koʻrsatadi. Lekin modelning ichidagi weightlar boshida random boʻladi. Random — tasodifiy. Tasodifiy weightlar bilan model hali rasmni toʻgʻri tushunmaydi.

Masalan, cat vs dog model architecture qurildi. Modelda convolutional layerlar, pooling layerlar, fully connected layer va softmax bor. Lekin model hali train qilinmagan boʻlsa, u mushuk va itni ajrata olmaydi. Chunki u hali mushuk qulogʻi, it tumshugʻi, hayvon yuz shakli kabi featurelarni oʻrganmagan boʻladi.

Architecture bilan modelda faqat quyidagi narsa boʻladi:

qatlamlar bor

weightlar bor

input va output yoʻli bor

hisoblash structure bor

Lekin quyidagilar hali yetishmaydi:

data
label
training
loss
backpropagation
optimizer
learning rate
epoch va batch
evaluation

Data yetishmaydi

Data — model o'rganadigan ma'lumot. Image Classificationda data rasmlardan iborat bo'ladi. Model mushukni tanishi uchun ko'p mushuk rasmlarini ko'rishi kerak. Itni tanishi uchun ko'p it rasmlarini ko'rishi kerak.

Agar data bo'lmasa, model o'rganadigan narsa bo'lmaydi.

Label yetishmaydi

Label — to'g'ri javob. Model rasmga qarab prediction qiladi, keyin label bilan solishtiriladi. Agar label bo'lmasa, model o'z predictioni to'g'ri yoki noto'g'ri ekanini bilmaydi.

Masalan:

Image: mushuk rasmi

Label: cat

Model rasmni dog deb aytsa, label orqali xato aniqlanadi.

Loss yetishmaydi

Loss — model xatosini oʻlchaydigan qiymat. Loss boʻlmasa, model qanchalik xato qilganini bilmaydi.

Masalan:

True label: Dog

Model prediction:

Dog: 0.30

Cat: 0.65

Bird: 0.05

Bu holatda model xato qilgan. Loss shu xatoni son sifatida oʻlchaydi. Cross-Entropy Loss bunday classification vazifalarida koʻp ishlatiladi.

Cross-Entropy Loss — haqiqiy class probabilitysi asosida xato hisoblaydigan loss function.

Backpropagation yetishmaydi

Backpropagation — lossdan kelgan xatoni orqaga tarqatib, gradientlarni hisoblaydigan jarayon. Gradient — weight qaysi yoʻnalishda oʻzgarishi kerakligini bildiradi.

Backpropagation boʻlmasa, model xato qilganini bilishi mumkin, lekin qaysi weightni qanday tuzatish kerakligini bilmaydi.

Optimizer yetishmaydi

Optimizer — gradientlardan foydalanib weightlarni yangilaydigan algoritm. Optimizer bo'lmasa, model weightlari o'zgarmaydi. Weightlar o'zgarmasa, model o'rganmaydi.

Optimizer misollari:

SGD

Adam

AdamW

RMSprop

SGD — Stochastic Gradient Descent. Adam — deep learningda ko'p ishlatiladigan optimizer.

Learning Rate yetishmaydi

Learning Rate — weight update qadamining kattaligi. Agar learning rate bo'lmasa yoki noto'g'ri tanlansa, model yaxshi o'rgana olmaydi.

Juda kichik learning rate:

model juda sekin o'rganadi

Juda katta learning rate:

model tebranadi yoki diverge bo'ladi

Diverge — trainingning beqaror bo'lib, loss kamaymay ketishi.

Epoch va Batch yetishmaydi

Epoch — model butun training datasetni bir marta to‘liq ko‘rib chiqishi.

Batch — datasetning kichik qismi.

Model o‘rganishi uchun dataset batchlarga bo‘linadi, har batchda training step bajariladi, barcha batchlar tugagandan keyin 1 epoch bo‘ladi.

Model o‘rganishi uchun to‘liq jarayon

Model o‘rganishi uchun quyidagi jarayon kerak:

```
Data + Label
↓
Architecture
↓
Forward Pass
↓
Prediction
↓
Loss
↓
Backpropagation
↓
Optimizer Step
↓
Updated Weights
↓
Repeat
```

Forward Pass — input modeldan o'tib prediction chiqaradi.

Prediction — model taxmini.

Loss — xatoni o'lchaydi.

Backpropagation — gradientlarni hisoblaydi.

Optimizer Step — weightlarni yangilaydi.

Updated Weights — yangilangan weightlar.

Repeat — jarayon takrorlanadi.

Model faqat architecture bilan o'rganmaydi. Architecture modelning tanasi bo'lsa, training va optimization uning o'rganish jarayoni hisoblanadi.

Qisqa javob:

Yo'q, model faqat architecture bilan o'rganib qolmaydi.

O'rganish uchun data, label, loss, backpropagation, optimizer, learning rate, epoch va batch kerak bo'ladi.

5. Agar model noto'g'ri javob bersa, uni qanday yaxshilash mumkin?

Agar model noto'g'ri javob bersa, uni yaxshilash uchun avval xatoni qayerdan kelayotganini aniqlash kerak. Model xatosi har doim faqat model architecture yomonligidan bo'lmaydi. Ba'zan data noto'g'ri bo'ladi, label xato bo'ladi, preprocessing noto'g'ri bajariladi, learning rate mos bo'lmaydi yoki model overfitting qilgan bo'ladi.

Modelni yaxshilash uchun quyidagi yo‘llar ishlatiladi.

1. Data sifatini tekshirish

Model noto‘g‘ri javob bersa, birinchi navbatda dataset tekshiriladi.

Dataset — model o‘rganadigan rasmlar to‘plami.

Tekshiriladigan narsalar:

rasmlar ochiladimi?

rasmlar to‘g‘ri class papkada turibdimi?

noto‘g‘ri label yo‘qmi?

buzilgan rasm yo‘qmi?

xira rasm juda ko‘p emasmi?

bir xil rasm juda ko‘p takrorlanmaganmi?

Masalan, cat papkasida it rasmi bo‘lsa, model chalkashadi. Chunki u it rasmiga cat labeli bilan o‘rganadi.

Noto‘g‘ri data modelni noto‘g‘ri o‘rgatadi. Shuning uchun modelni yaxshilashda data cleaning juda muhim.

Data Cleaning — ma’lumotni tozalash. Bunda xato, takroriy, buzilgan yoki noto‘g‘ri rasm olib tashlanadi.

2. Ko‘proq data yig‘ish

Model kam data bilan train qilingan bo‘lsa, yaxshi generalization qilmasligi mumkin. Generalization — modelning oldin ko‘rmagan yangi rasmlarda ham yaxshi ishlashi.

Masalan, model faqat 50 ta mushuk va 50 ta it rasmi bilan train qilingan bo'lsa, real hayotdagi turli mushuk va itlarni tanishda qiynalishi mumkin.

Yaxshi model uchun har classda yetarli rasm bo'lishi kerak:

cat: ko'p va turli rasm

dog: ko'p va turli rasm

Rasmlar turli holatda bo'lishi kerak:

turli yorug'lik

turli fon

turli burchak

turli masofa

turli rang

turli sifat

Bu modelni real hayotga yaqinlashtiradi.

3. Data Augmentation qilish

Data Augmentation — mavjud rasmlardan yangi ko'rinishlar yaratish. Augmentation modelni turli holatlarga moslashishga o'rgatadi.

Augmentation misollari:

Random rotation

Horizontal flip

Random crop

Brightness change

Contrast change

Color jitter

Zoom

Blur

Random rotation — rasmni tasodifiy burish.

Horizontal flip — rasmni chap-o'ngga ag'darish.

Random crop — rasmning tasodifiy qismini kesib olish.

Brightness change — yorug'likni o'zgartirish.

Contrast change — contrastni o'zgartirish.

Color jitter — ranglarni biroz o'zgartirish.

Blur — xiralashtirish.

Augmentation modelga bir xil obyekt turli ko'rinishda ham o'sha classga tegishli ekanini o'rgatadi.

4. Preprocessingni to'g'rilash

Preprocessing noto'g'ri bo'lsa, model noto'g'ri o'rganishi mumkin.

Preprocessing — rasmni modelga berishdan oldin tayyorlash.

Tekshiriladigan narsalar:

rasm o'lchami to'g'rimi?

normalization to'g'rimi?

RGB/BGR adashmaganmi?

train va inference preprocessing bir xilmi?

RGB — Red, Green, Blue rang kanallari. OpenCV ko‘pincha BGR formatda o‘qiydi. BGR — Blue, Green, Red. Agar model RGB kutsa, lekin rasm BGR bo‘lsa, ranglar noto‘g‘ri chiqadi.

Trainingda qanday preprocessing qilingan bo‘lsa, inference paytida ham shunday bo‘lishi kerak. Aks holda model noto‘g‘ri prediction qilishi mumkin.

5. Learning Rate sozlash

Learning Rate model o‘rganish tezligini belgilaydi. Agar learning rate noto‘g‘ri bo‘lsa, model yaxshi o‘rgana olmaydi.

Juda kichik learning rate:

loss juda sekin kamayadi
training uzoq vaqt oladi
model yetarlicha o‘rganmasligi mumkin

Juda katta learning rate:

loss tebranadi
model minimumdan sakrab o‘tadi
training diverge bo‘lishi mumkin

Yaxshi learning rate tanlash uchun tajriba qilish kerak. Masalan:

0.001

0.0005

0.0001

Learning Rate Scheduler ham ishlatilishi mumkin. Scheduler training davomida learning rateni o'zgartirib boradi.

6. Optimizer almashtirish

Optimizer model weightlarini yangilaydi. Agar optimizer mos bo'lmasa, training sekin yoki beqaror bo'lishi mumkin.

Optimizer turlari:

SGD

Adam

AdamW

RMSprop

Adam ko'p holatda boshlash uchun qulay. SGD ba'zan yaxshi generalization berishi mumkin. AdamW regularization bilan yaxshi ishlaydi.

Model noto'g'ri javob berayotgan bo'lsa, optimizer va uning learning rate qiymatini sinab ko'rish mumkin.

7. Model architecture'ni kuchaytirish

Agar model juda sodda bo'lsa, murakkab featurelarni o'rgana olmaydi. Bu underfitting bo'lishi mumkin.

Underfitting — model data ichidagi patternlarni yetarlicha o'rgana olmasligi.

Yechimlar:

chuqurroq CNN ishlatish
pretrained model ishlatish
ResNet yoki EfficientNet ishlatish
model capacityni oshirish

Model capacity — modelning oʻrganish quvvati. Juda kichik model murakkab vazifani bajara olmasligi mumkin.

Pretrained model — oldin katta datasetda train qilingan tayyor model. Masalan, ImageNetda train qilingan ResNet. Pretrained modeldan foydalanish transfer learning deyiladi.

Transfer Learning — oldin oʻrgangan bilimni yangi vazifaga koʻchirish.

8. Overfittingni kamaytirish

Agar model train datasetda yaxshi, validation datasetda yomon ishlasa, overfitting bor boʻlishi mumkin.

Overfitting belgisi:

train accuracy yuqori
validation accuracy past
train loss past
validation loss yuqori

Overfittingni kamaytirish yoʻllari:

data augmentation
dropout
weight decay
early stopping

ko'proq data
modelni kichraytirish

Dropout — training paytida ayrim neuronlarni vaqtincha o'chirib turish.

Weight decay — weightlarning juda katta bo'lib ketishini cheklash.

Early stopping — validation natija yomonlashsa trainingni to'xtatish.

9. Class imbalance muammosini hal qilish

Class imbalance — classlar soni juda noteng bo'lishi.

Masalan:

cat: 5000 ta rasm

dog: 200 ta rasm

Bunday holatda model ko'proq cat classini yaxshi o'rganadi, dog classida xato qilishi mumkin.

Yechimlar:

kam classga ko'proq rasm yig'ish

class weight ishlatish

oversampling

undersampling

augmentation

Class weight — kam classga loss hisoblashda ko'proq og'irlik berish.

Oversampling — kam classdagi rasmlarni ko'paytirib ishlatish.

Undersampling — ko‘p classdagi rasmlarni kamaytirish.

10. Error Analysis qilish

Error Analysis — model xatolarini tahlil qilish. Model qaysi rasmlarda xato qilayotganini ko‘rish juda muhim.

Masalan, model noto‘g‘ri topgan rasmlar alohida ajratiladi:

cat rasmini dog deb topgan
dog rasmini cat deb topgan
xira rasmda xato qilgan
qorong‘i rasmda xato qilgan
obyekt kichik bo‘lganda xato qilgan

Keyin xato sababi aniqlanadi:

rasm sifati past
label noto‘g‘ri
classlar o‘xshash
data kam
preprocessing noto‘g‘ri
model juda sodda

Error Analysis modelni maqsadli yaxshilashga yordam beradi.

11. Confusion Matrix ko‘rish

Confusion Matrix — model qaysi classni qaysi class bilan adashtirayotganini ko‘rsatadigan jadval.

Masalan:

Haqiqiy cat → model dog deb topgan

Haqiqiy dog → model cat deb topgan

Confusion Matrix orqali qaysi classda xato ko‘p ekanini ko‘rish mumkin.

Agar bitta classda xato ko‘p bo‘lsa, o‘sha class uchun ko‘proq data, augmentation yoki class weight kerak bo‘lishi mumkin.

12. Modelni qayta train qilish

Yuqoridagi o‘zgarishlardan keyin model qayta train qilinadi.

Retraining — modelni qayta o‘qitish.

Retraining jarayoni:

xatolar tahlil qilinadi

data tozalanadi

yangi data qo‘shiladi

preprocessing to‘g‘rilanadi

model qayta train qilinadi

test natija baholanadi

Modelni yaxshilash bir martalik jarayon emas. Real projectlarda model bir necha marta train qilinadi, baholanadi va yaxshilanadi.

Qisqa javob:

Model noto‘g‘ri javob bersa, avval xato sababi topiladi.

Keyin data, preprocessing, augmentation, learning rate, optimizer, architecture va regularization

orqali model yaxshilanadi.

6. SML vs CV architecture'larini yonma-yon chizish, farqli va o'xshash tomonlari

SML va CV architecture'lari umumiy jihatdan o'xshash: ikkalasida ham data olinadi, preprocessing qilinadi, model train qilinadi, evaluation qilinadi va deployment qilinadi. Lekin ishlaydigan data turi, preprocessing usuli, model architecture va output ko'rinishi farq qiladi.

SML — Supervised Machine Learning. O'zbekcha ma'nosi "nazoratli mashinali o'rganish". SMLda dataset ichida input featurelar va label bo'ladi.

CV — Computer Vision. O'zbekcha ma'nosi "kompyuter ko'rishi". CV rasm va video bilan ishlaydi.

Architecture bu yerda to'liq model yasash ketma-ketligi sifatida olinadi.

SML architecture

SML odatda tabular data bilan ishlaydi. Tabular data — jadval ko'rinishidagi ma'lumot. Masalan, CSV, Excel yoki SQL jadvali.

SML architecture:

Raw Tabular Data

↓

Data Cleaning

↓

Feature Engineering
↓
Encoding
↓
Scaling
↓
Train / Validation / Test Split
↓
ML Model
↓
Prediction
↓
Evaluation
↓
Deployment

Raw Tabular Data — hali tozalanmagan jadval ma'lumot.

Data Cleaning — missing value, duplicate, outlier kabi muammolarni tuzatish.

Missing value — bo'sh qiymat.

Duplicate — takroriy qator.

Outlier — juda noodatiy yoki chetda turgan qiymat.

Feature Engineering — yangi foydali ustunlar yaratish. Masalan, `income_per_age`, `total_spend`, `purchase_frequency`.

Encoding — matnli kategoriyalarni raqamga aylantirish. Masalan, `city = Seoul` qiymatini raqamga aylantirish.

Scaling — sonli ustunlarni bir xil scale'ga keltirish. Masalan, StandardScaler yoki MinMaxScaler.

ML Model — Logistic Regression, Random Forest, XGBoost, LightGBM, CatBoost kabi model.

Prediction — model taxmini. Masalan, approved yoki rejected.

Evaluation — modelni metriclar bilan baholash. Masalan, accuracy, precision, recall, F1-score.

Deployment — modelni real tizimga chiqarish.

CV architecture

CV rasm yoki video bilan ishlaydi. Image Classification uchun architecture quyidagicha bo'ladi:

Raw Images

↓

Data Preparation

↓

Preprocessing

↓

Augmentation

↓

Train / Validation / Test Split

↓

CNN / Pretrained Model

↓

Forward Pass

↓

Loss

↓

Backpropagation

↓

Optimizer Step

↓

Evaluation

↓

Deployment / Inference

Raw Images — xom rasmlar.

Data Preparation — rasmlarni class papkalarga joylash, label tayyorlash.

Preprocessing — resize, normalize, tensor formatga o'tkazish.

Augmentation — rasmni aylantirish, flip qilish, brightness va contrast o'zgartirish.

CNN / Pretrained Model — Computer Vision modeli. Masalan, CNN, ResNet, EfficientNet, MobileNet.

Forward Pass — rasm modeldan o'tib prediction beradi.

Loss — model xatosini o'lchaydi.

Backpropagation — xatoni orqaga tarqatib gradientlarni hisoblaydi.

Optimizer Step — weightlarni yangilaydi.

Evaluation — accuracy, precision, recall, F1-score, confusion matrix bilan baholash.

Deployment / Inference — modelni real ishlatish.

SML va CV yonma-yon chizma

SML Architecture
Architecture

CV

Raw Tabular Data
/ Videos

Raw Images



Data Cleaning
Preparation

Data



Feature Engineering
Preprocessing



Encoding
Augmentation



Scaling
Val / Test Split

Train /



Train / Val / Test Split
Pretrained Model

CNN /



ML Model
Pass

Forward



Prediction

Loss



Evaluation
Backpropagation

↓
Deployment
Step

↓
Optimizer
↓
Evaluation
↓
Deployment

/ Inference

Bu chizmada SML ko‘proq jadval ustunlari bilan ishlaydi. CV esa rasm, pixel, tensor va CNN bilan ishlaydi.

SML va CV o‘xshash tomonlari

SML va CV o‘rtasida bir nechta o‘xshashlik bor.

1. Ikkalasida ham data kerak

SMLda data jadval bo‘ladi. CVda data rasm yoki video bo‘ladi. Lekin ikkalasida ham model o‘rganishi uchun data kerak.

SML → tabular data

CV → image / video data

2. Ikkalasida ham label kerak

Supervised learningda label bo‘lishi kerak. Label — to‘g‘ri javob.

SML misol:

customer data → churn yoki not churn

Churn — mijoz ketib qolishi.

CV misol:

image → cat yoki dog

3. Ikkalasida ham preprocessing kerak

SMLda preprocessing missing value, encoding, scaling orqali bajariladi.

CVda preprocessing resize, normalize, tensor conversion orqali bajariladi.

Tensor conversion — rasmni tensor ko‘rinishiga o‘tkazish. Tensor — ko‘p o‘lchamli raqamlar massivi.

4. Ikkalasida ham model train qilinadi

SMLda Random Forest, XGBoost, Logistic Regression train qilinadi.

CVda CNN, ResNet, EfficientNet train qilinadi.

5. Ikkalasida ham evaluation bor

Ikkalasida ham model natijasi metriclar bilan baholanadi.

Metriclar:

Accuracy

Precision

Recall

F1-score

Confusion Matrix

Accuracy — umumiy to‘g‘ri javoblar ulushi.

Precision — model bir class deb aytganlar ichida nechta to‘g‘ri ekanini bildiradi.

Recall — haqiqiy classlardan nechta topilganini bildiradi.

F1-score — precision va recall o‘rtasidagi muvozanat.

Confusion Matrix — model qaysi classni qaysi class bilan adashtirganini ko‘rsatadigan jadval.

6. Ikkalasida ham deployment bo‘ladi

Model faqat notebookda qolib ketmasligi kerak. SML ham, CV ham API, web app yoki product ichida ishlatilishi mumkin.

SML va CV farqli tomonlari

1. Data turi farq qiladi

SML odatda jadval bilan ishlaydi:

`age, salary, city, income, score`

CV esa rasm va video bilan ishlaydi:

`image pixels, RGB channels, frames`

Frame — videodagi bitta rasm.

2. Feature yaratish farq qiladi

SMLda feature ko‘pincha odam tomonidan yaratiladi. Masalan:

```
total_spend = product_1 + product_2 + product_3  
age_group = age bo'yicha guruh
```

CVda featurelarni modelning o'zi o'rganadi. CNN rasm ichidan qirra, chiziq, shakl, obyekt qismlarini ajratadi.

3. Model turi farq qiladi

SML modellari:

Logistic Regression

Decision Tree

Random Forest

XGBoost

LightGBM

CatBoost

SVM

KNN

CV modellari:

CNN

ResNet

EfficientNet

MobileNet

Vision Transformer

YOLO

U-Net

YOLO — Object Detection uchun ishlatiladi.

U-Net — Segmentation uchun ishlatiladi.

4. Hisoblash quvvati farq qiladi

SML ko‘pincha CPUda ham yaxshi ishlaydi. CPU — Central Processing Unit, markaziy protsessor.

CV odatda GPU talab qiladi. GPU — Graphics Processing Unit, grafik protsessor. Rasm va video katta hajmli bo‘lgani uchun GPU trainingni tezlashtiradi.

5. Preprocessing farq qiladi

SML preprocessing:

- missing value
- encoding
- scaling
- outlier handling

CV preprocessing:

- resize
- normalize
- crop
- augmentation
- tensor conversion

6. Output ko‘rinishi farq qiladi

SML output:

- class
- probability
- numeric prediction

CV output:

class label
bounding box
segmentation mask
OCR text
pose keypoints

Bounding box — obyekt atrofidagi to'rtburchak ramka.

Segmentation mask — pixel darajasida ajratilgan hudud.

OCR text — rasmdan o'qilgan matn.

Pose keypoints — tana nuqtalari. Masalan, yelka, tirsak, tizza.

SML vs CV taqqoslash jadvali

Taqqoslash	SML	CV
To'liq nomi	Supervised Machine Learning	Computer Vision
Ma'lumot turi	Jadval, CSV, Excel	Rasm, video, frame
Input	Feature ustunlari	Pixel / tensor
Feature	Ko'pincha odam yaratadi	Model o'zi ajratadi
Preprocessing	Missing, encoding, scaling	Resize, normalize, augmentation
Model	RF, XGBoost, LR, SVM	CNN, ResNet, EfficientNet
Hisoblash	CPU yetishi mumkin	Ko'pincha GPU kerak
Output	Class yoki son	Class, box, mask, text
Metric	Accuracy, F1, ROC-AUC	Accuracy, F1, IoU, mAP
Deployment	API, dashboard, app	API, camera, mobile, edge

IoU — Intersection over Union. Object Detection va Segmentationda ishlatiladigan metric. mAP — mean Average Precision. Object Detectionda model sifatini baholash metrici.

SML va CV umumiy xulosa

SML va CV ikkalasi ham Machine Learningning muhim yo'nalishlari hisoblanadi. SML ko'proq jadval ma'lumotlar bilan ishlaydi. CV esa rasm va videolar bilan ishlaydi. Ikkalasida ham data tayyorlanadi, model train qilinadi, natija baholanadi va productga chiqariladi.

Eng katta farq data turida va feature extraction usulida. SMLda featurelarni ko'pincha odam yaratadi. CVda esa model featurelarni rasm ichidan o'zi o'rganadi. Shu sababli CVda CNN va Deep Learning modellari ko'p ishlatiladi.

Qisqa xulosa:

SML → jadval data → feature engineering → ML model
→ prediction

CV → rasm/video data → feature extraction → CNN
model → prediction

Ikkalasining umumiy maqsadi bir xil: data'dan o'rganib, yangi ma'lumotga to'g'ri prediction qilish.